

# Track A config ( `configs/trackA.yaml` ) — what each key changes (Stage A vs Stage B)

---

This note maps every key in `local_extraction/configs/trackA.yaml` to:

- **Which stage reads it** (Stage A / Stage B / tooling)
- **What it changes** (outputs, metrics, runtime)
- **Why it changes it** (mechanism in the code)
- **How to use it** (typical edits and expected result changes)

It's written against the current local runners:

- Stage A: `local_extraction/trackA/trackA_stageA/trackA_stageA.py`
  - Stage B: `local_extraction/trackA/trackA_stageB/trackA_stageB.py`
- 

## How config is loaded

- Track A loads `trackA.yaml`, which inherits shared defaults from `base.yaml` via `_base_: "base.yaml"`.
- The loader supports variable interpolation like `${paths.local_extraction}`.

Important detail: the Stage A/B scripts look for some path keys that are **not present in** `base.yaml` ( `paths.extracted_frames`, `paths.yolo_labels`, `paths.runs`, `paths.manifests` ). If they're missing, the scripts fall back to defaults like `local_extraction/{version}/extracted_frames`.

For strict reproducibility (and to make paths explicit), it's recommended to add these keys to `trackA.yaml` under `paths:`.

---

## Global / shared keys

### `version`

- **Used by:** Stage A and Stage B
- **Effect:** Selects the dataset version folder under `local_extraction/` when path overrides are not provided.
- **Why:** Both scripts default frames/labels/manifests to `local_extraction/{version}/...`.
- **Changing it changes results by:** Changing the *input dataset* (different frames/labels/manifests) → produces different candidates and different recall/IoU stats.

### `paths.*` (in this Track A config)

Track A's scripts mainly care about these logical paths:

### `paths.local_extraction`

- **Used by:** config interpolation (e.g., YOLO weights path)

- **Effect:** Acts as the base variable in `${paths.local_extraction}/...`.
- **Why:** Used to build absolute/relative paths inside config.
- **Changing it changes results by:** Usually **does not** change results directly; it changes *where files are read from*. If it points to different weights/data, outputs change.

**(Recommended)** `paths.extracted_frames`

- **Used by:** Stage A and Stage B
- **Effect:** Where frames are read from.
- **Why:** Both scripts construct image paths as `${frames_root}/${uid}/${frame:07d}.jpg`.
- **Changing it changes results by:** If the frames differ, both the detector outputs and the crop outputs change.

**(Recommended)** `paths.yolo_labels`

- **Used by:** Stage A (oracle mode) and Stage B (evaluation)
- **Effect:** Where label `.txt` files are read from.
- **Why:** Oracle mode reads labels as proposals; Stage B uses labels to compute recall@K and IoU statistics.
- **Changing it changes results by:**
  - In **oracle mode**, changes the candidate boxes (StageA output changes).
  - In **eval\_with\_labels=true**, changes metrics (StageB recall/IoU) even if candidates are unchanged.

**(Recommended)** `paths.runs`

- **Used by:** Stage A and Stage B
- **Effect:** Base folder where run directories are created.
- **Why:** Both scripts write outputs to `${paths.runs}/Track_A/<run_prefix>_<timestamp>/...`.
- **Changing it changes results by:** Output location changes; numeric outputs are the same, but Stage B "auto-detect latest Stage A" behavior can change (because it scans the runs folder).

**(Used optionally)** `paths.manifests`

- **Used by:** Stage B
- **Effect:** Default directory used to auto-resolve head manifest filenames when not explicitly provided.
- **Why:** Stage B tries `${paths.manifests}` then falls back to `local_extraction/{version}/manifests`.
- **Changing it changes results by:** If it points to different train/val manifests, Stage B's semantic labeling and head output files change.

## Stage A keys ( `stage_a.*` ) — candidate generation

Stage A writes:

- `candidates.jsonl` (the main "result" of Stage A)
- `summary.json` (records the settings and runtime)

`stage_a.mode`

- **Used by:** Stage A
- **Effect:** Chooses how candidate boxes are produced.
  - `"yolo"` : run Ultralytics YOLO inference
  - `"oracle"` : use existing YOLO label files as proposals
- **Why:** The code path switches between `yolo_detect_one()` vs `oracle_boxes()` .
- **Changing it changes results by:** Producing a completely different candidate set:
  - `oracle` is an "upper bound" (candidates are the GT boxes, up to K)
  - `yolo` depends on model weights, thresholds, image size

### stage\_a.k

- **Used by:** Stage A (and indirectly Stage B)
- **Effect:** Maximum number of boxes kept per image (top-K by confidence in YOLO mode).
- **Why:** Stage A truncates the candidate list to `K` .
- **Changing it changes results by:**
  - Higher K → more candidates per image → Stage B creates more crops and typically higher recall@K
  - Lower K → fewer crops/faster Stage B but lower recall

### stage\_a.last\_frame\_only

- **Used by:** Stage A
- **Effect:** Whether each UID contributes only its last frame or all frames.
- **Why:** The image iterator yields `imgs[-1]` for each uid folder when `true` .
- **Changing it changes results by:**
  - `true` → fewer images → faster, matches "last-frame" STA assumption
  - `false` → many more images → more candidates, different dataset distribution → different metrics

### stage\_a.max\_images

- **Used by:** Stage A
- **Effect:** Caps the number of images processed.
- **Why:** Stage A truncates the images list to the first `max_images` .
- **Changing it changes results by:** Not a model change; it changes *which subset* you processed → metrics and output size differ.

### stage\_a.yolo.weights

- **Used by:** Stage A (YOLO mode only)
- **Effect:** Selects the YOLO checkpoint.
- **Why:** The Ultralytics `YOLO(weights)` object defines the detector.
- **Changing it changes results by:** This is usually the **biggest lever** for Stage A quality. Different weights → different boxes/confidences → different recall/IoU downstream.

### stage\_a.yolo.imgsz

- **Used by:** Stage A (YOLO mode only)
- **Effect:** Input inference resolution (Ultralytics `imgsz` ).

- **Why:** Changes the scale at which the model sees the image; affects detection of small objects and runtime.
- **Changing it changes results by:** Larger `imgsz` tends to improve localization/recall for small objects but costs time/VRAM.

### `stage_a.yolo.conf_thresh`

- **Used by:** Stage A (YOLO mode only)
- **Effect:** Filters detections below this confidence.
- **Why:** Ultralytics does thresholding before returning boxes.
- **Changing it changes results by:**
  - Lower threshold → more boxes (often more false positives) → with top-K, you can increase recall but may add noise
  - Higher threshold → fewer boxes → faster Stage B, may reduce recall

### `stage_a.yolo.nms_iou`

- **Used by:** Stage A (YOLO mode only)
- **Effect:** NMS suppression strength.
- **Why:** NMS removes overlapping boxes; higher IoU threshold suppresses less.
- **Changing it changes results by:**
  - Higher `nms_iou` → keeps more overlapping boxes (more duplicates)
  - Lower `nms_iou` → fewer overlaps (can drop alternate hypotheses)

### `stage_a.oracle.label_space`

- **Used by:** Stage A (oracle mode) and Stage B (evaluation)
- **Effect:** Chooses label folder layout: `clips` vs `videos` .
- **Why:** Label files are read from `${labels_root}/${label_space}/${uid}/${frame}.txt` .
- **Changing it changes results by:**
  - If set wrong, oracle candidates can be empty and Stage B evaluation can show low recall (missing GT files).

### `stage_a.output.save_per_image_csv`

- **Used by:** Stage A
- **Effect:** Writes per-image CSVs of candidates.
- **Why:** Pure output option.
- **Changing it changes results by:** No numerical change; increases disk usage and makes debugging easier.

### `stage_a.output.run_prefix`

- **Used by:** Stage A
- **Effect:** Prefix of the Stage A run directory name.
- **Why:** Used to build `${run_prefix}_${timestamp}` .
- **Changing it changes results by:** No numerical change; helps keep runs separate and affects Stage B auto-detect (it scans `runs/Track_A` for any folder containing `candidates.jsonl` ).

## Stage B keys ( `stage_b.*` ) — cropping, evaluation, head-manifest prep

Stage B writes:

- `crops/` (ROI images)
- `manifest.csv` + `manifest.jsonl`
- `summary.json` (Stage B summary)
- Optionally `head_train.jsonl` / `head_val.jsonl`
- Optionally patches Stage A's `summary.json` with `stageB_metrics`

### `stage_b.candidates_source`

- **Used by:** Stage B
- **Effect:** Selects which `candidates.jsonl` to read.
- **Why:** If `null`, Stage B picks the most recently modified `candidates.jsonl` under `local_extraction/runs/Track_A`.
- **Changing it changes results by:** Pointing to a different Stage A run gives different crops and different recall metrics.

### `stage_b.crop_size`

- **Used by:** Stage B
- **Effect:** Resizes saved crops to a fixed size, e.g. `[256,256]`, or keeps native crop size if `null`.
- **Why:** Stage B uses PIL `roi.resize(CROP_SIZE)` before saving.
- **Changing it changes results by:**
  - The crop pixels change (different downstream model input)
  - Stage B metrics like recall@K do **not** change (they're computed on box geometry), but any later model that consumes crops will.

### `stage_b.keep_top_n`

- **Used by:** Stage B
- **Effect:** Limits how many of the Stage A candidates are cropped/evaluated per image.
- **Why:** Stage B stops after `keep_top_n` boxes.
- **Changing it changes results by:**
  - Lower value reduces disk/time and reduces recall@N (because you're discarding candidates)
  - Higher value increases crops/time and can increase recall

### `stage_b.eval_with_labels`

- **Used by:** Stage B
- **Effect:** Enables recall@K / best-IoU evaluation using YOLO label files.
- **Why:** When true, Stage B loads GT boxes and computes hit/best IoU.
- **Changing it changes results by:** It changes **what gets reported**, not the crops themselves:
  - `false` → no recall metrics computed
  - `true` → recall metrics + histogram available (and can be written back into Stage A summary)

### `stage_b.iou_thresh`

- **Used by:** Stage B
- **Effect:** IoU threshold  $\tau$  used for defining a “hit” (recall@K) and a “positive” ROI (for head files).
- **Why:** Both the hit computation and `is_positive` are `IoU >=  $\tau$` .
- **Changing it changes results by:**
  - Lower  $\tau$  → higher recall@K and more ROIs labeled positive
  - Higher  $\tau$  → stricter; recall drops, fewer positives

### `stage_b.manifests.use_clip_manifest`

- **Used by:** Stage B
- **Effect:** Controls default file naming when `train/val` are not specified (clip vs video).
- **Why:** Stage B chooses between `head*_clip.json` vs `head*_video.json` when building defaults.
- **Changing it changes results by:** It changes which semantic labels (verb/noun/ttc) are attached to ROIs.

### `stage_b.manifests.train` / `stage_b.manifests.val`

- **Used by:** Stage B
- **Effect:** Paths to semantic manifests indexed by `(uid, frame)`.
- **Why:** Stage B joins candidates to manifests to attach `verb_id`, `noun_id`, `ttc`, and compute `is_positive`.
- **Changing it changes results by:** Different manifests → different semantic fields and different `head_train/head_val` content.

### `stage_b.output.write_head_train_val`

- **Used by:** Stage B
- **Effect:** Writes consolidated `head_train.jsonl` and `head_val.jsonl` in the Stage B run.
- **Why:** Enables writing those files and counts rows written.
- **Changing it changes results by:** No change to candidates/crops; it changes whether the training files exist.

### `stage_b.output.write_semantics_in_manifest`

- **Used by:** Stage B
- **Effect:** Adds semantic fields (`verb_id`, `noun_id`, `ttc`, `is_positive`, `iou`) into `manifest.jsonl` rows.
- **Why:** Stage B optionally augments manifest rows when semantics are available.
- **Changing it changes results by:** Changes what’s recorded for later analysis; crops/recall stay the same.

### `stage_b.output.write_back_to_stagea_summary`

- **Used by:** Stage B
- **Effect:** Writes Stage B’s recall metrics into the Stage A `summary.json` next to the candidates.
- **Why:** Convenience feature to keep “detector run + metrics” in a single file.
- **Changing it changes results by:** No numerical change; it changes where metrics are stored.

### `stage_b.output.run_prefix`

- **Used by:** Stage B
  - **Effect:** Prefix of Stage B run directory name.
  - **Why:** Used to build output folder name.
  - **Changing it changes results by:** Output location changes; no numerical change.
- 

## K-sweep keys ( `k_sweep.*` ) — experiment tooling

### `k_sweep.enabled`

- **Used by:** tooling scripts (not the plain Stage A/B runners)
- **Effect:** Intended to toggle K sweeps.
- **Why:** Sweep scripts iterate over multiple K values.
- **Changing it changes results by:** Not applicable to a single Stage A/B run; it affects whether your sweep automation runs.

### `k_sweep.k_values`

- **Used by:** tooling scripts
  - **Effect:** List of K values to evaluate.
  - **Changing it changes results by:** Produces multiple runs/metrics at different K, letting you pick a K that balances recall vs cost.
- 

## Smoke test keys ( `smoke_test.*` ) — validation tooling

These are designed for a separate smoke-test harness (not used by Stage A/B runners directly).

### `smoke_test.enabled`

- **Effect:** Whether smoke testing should run.

### `smoke_test.max_samples`

- **Effect:** Max samples for smoke tests.

### `smoke_test.validate_pycache`

- **Effect:** Sanity check on stale bytecode issues.

### `smoke_test.expected_outputs`

- **Effect:** What files a smoke test expects to find.

Changing these typically does **not** change the Stage A/B results; it changes whether validations pass and how big the smoke run is.

---

## Quick “what should I change?” recipes

1) Improve recall@K (more hits)

- Increase `stage_a.k` (e.g., 6 → 10)
- Lower `stage_a.yolo.conf_thresh` slightly (e.g., 0.05 → 0.02)
- Consider increasing `stage_a.yolo.imgsz` if you can afford runtime

2) Reduce runtime / disk

- Keep `stage_a.last_frame_only: true`
- Reduce `stage_a.k` and/or set `stage_a.max_images`
- Set `stage_b.keep_top_n` lower (e.g., 6 → 3)
- Disable `stage_a.output.save_per_image_csv`

3) Make Stage B evaluate a specific detector run

- Set `stage_b.candidates_source` to that run's `candidates.jsonl`

4) Change what is considered a "positive" ROI

- Adjust `stage_b.iou_thresh` (this directly changes `is_positive` and recall definition)

Summary

- Stage A config primarily controls **which frames** are processed and **which boxes** are proposed ( `mode` , `k` , YOLO weights/thresholds). This directly changes `candidates.jsonl` , and therefore everything Stage B sees.
- Stage B config primarily controls **how many proposals are used**, **how crops are written**, and **how hits/positives are defined** ( `keep_top_n` , `crop_size` , `eval_with_labels` , `iou_thresh` ). This changes crop outputs and reported metrics, and can optionally write metrics back into the Stage A summary.
- The largest "quality" lever is usually `stage_a.yolo.weights` (model) plus the "recall vs cost" tradeoff knobs `stage_a.k` and `stage_a.yolo.conf_thresh` .
- For reproducibility, prefer explicit `stage_b.candidates_source` (avoid "latest run" auto-detect) and explicitly set the `paths.*` keys instead of relying on fallbacks.

Numerical examples (illustrative comparisons)

These tables use **illustrative numbers** to show typical directional effects. Your actual values depend on data, weights, and label quality.

Example A — Changing `K` (more proposals → more crops → higher recall)

Assume Stage A processes 2,323 images (last frame only). Stage B uses `keep_top_n = null` (keeps all from Stage A).

| Config change             | Stage A: avg boxes/image | Stage B: crops written | Stage B: recall@K | Notes                                   |
|---------------------------|--------------------------|------------------------|-------------------|-----------------------------------------|
| <code>stage_a.k: 4</code> | ~1.6                     | ~3,700                 | ~0.62             | Faster Stage B, fewer chances to hit GT |



| Config change                 | Stage A: avg boxes/image | Stage B: crops written | Stage B: recall@K | Notes                                 |
|-------------------------------|--------------------------|------------------------|-------------------|---------------------------------------|
| <code>stage_a.k:</code><br>6  | ~2.2                     | ~5,100                 | ~0.68             | Similar to your recorded run behavior |
| <code>stage_a.k:</code><br>10 | ~3.1                     | ~7,200                 | ~0.74             | More disk/time; recall tends to rise  |

Example B — Changing YOLO confidence threshold (more detections, but noisier)

Assume `K=6` , same weights.

| Config change                                  | Stage A: avg boxes/image | Stage B: recall@6 | Mean best IoU | Notes                                        |
|------------------------------------------------|--------------------------|-------------------|---------------|----------------------------------------------|
| <code>stage_a.yolo.conf_thresh:</code><br>0.10 | ~1.4                     | ~0.61             | ~0.64         | Fewer low-confidence boxes; can lose recall  |
| <code>stage_a.yolo.conf_thresh:</code><br>0.05 | ~2.2                     | ~0.68             | ~0.62         | Baseline-ish                                 |
| <code>stage_a.yolo.conf_thresh:</code><br>0.02 | ~2.8                     | ~0.71             | ~0.59         | More candidates include more false positives |

Example C — Stage B limiting proposals ( `keep_top_n` ) vs Stage A `K`

If Stage A generates `K=10` but Stage B uses only the first N proposals, Stage B recall becomes **recall@N** even though Stage A computed 10.

| Config combo | Stage A K | Stage B keep_top_n | Crops/image (cap) | Expected recall metric | Why                              |
|--------------|-----------|--------------------|-------------------|------------------------|----------------------------------|
| A            | 10        | <code>null</code>  | up to 10          | recall@10              | Uses all proposals               |
| B            | 10        | 6                  | up to 6           | recall@6               | Discards lower-ranked candidates |
| C            | 6         | 6                  | up to 6           | recall@6               | Equivalent cap in both stages    |

Example D — Changing `iou_thresh` changes what counts as a hit/positive

This affects **Stage B metrics** and **head\_train/head\_val labeling**, not the crop pixels.

| Config change                            | “Hit” definition | Expected effect on recall@K | Expected effect on #positives | Typical use                             |
|------------------------------------------|------------------|-----------------------------|-------------------------------|-----------------------------------------|
| <code>stage_b.iou_thresh:</code><br>0.30 | IoU ≥ 0.30       | Higher                      | More positives                | Lenient diagnostic / early-stage tuning |

| Config change                         | “Hit” definition       | Expected effect on recall@K | Expected effect on #positives | Typical use                         |
|---------------------------------------|------------------------|-----------------------------|-------------------------------|-------------------------------------|
| <code>stage_b.iou_thresh: 0.50</code> | $\text{IoU} \geq 0.50$ | Baseline                    | Baseline                      | Standard object-detection threshold |
| <code>stage_b.iou_thresh: 0.75</code> | $\text{IoU} \geq 0.75$ | Lower                       | Fewer positives               | Strict localization experiments     |

Example E — Changing `crop_size` changes downstream inputs (not recall geometry)

| Config change                             | Crop file pixels | Disk/runtime                     | Stage B recall@K | Downstream impact               |
|-------------------------------------------|------------------|----------------------------------|------------------|---------------------------------|
| <code>stage_b.crop_size: null</code>      | Variable size    | Lower CPU resize cost            | Same             | Harder to batch for CNNs        |
| <code>stage_b.crop_size: [256,256]</code> | Fixed size       | More CPU, larger standardization | Same             | Easier for training/inference   |
| <code>stage_b.crop_size: [224,224]</code> | Fixed size       | Slightly smaller                 | Same             | Matches many ImageNet backbones |